

Chapter 4

Lexical Analysis in Practice

From token specifications to running scanners: NFA and DFA
scanning, language quirks, hand-written lexers and flex

Compiler Construction Module

Based on Lecture 4 (“Part 2: Lexical analysis”), expanded

How to read this chapter

Chapters 2 and 3 answered two questions: *what* is a token (Chapter 2), and *how* does a machine recognise one (Chapter 3, regular expression $\rightarrow$ NFA $\rightarrow$ DFA). Lecture 4 looks at the same subject from the point of view of someone who has to *ship* a scanner. It asks how the lexer sits next to the parser, how to run an NFA directly without ever building a DFA, how many token automata become one, what to do when nothing matches, what each choice costs in time and memory, which real languages break the whole model, and finally how to write a scanner by hand in C or generate one with `flex`.

Where the lecture repeats Chapter 3 (Thompson’s construction, subset construction, minimisation), this chapter gives the short version and points back. Where it adds something new, the chapter goes slowly. The new material is direct NFA simulation and its cost, the textbook (“Dragon book”) convention for Thompson’s construction, the catch-all rule, language quirks, the hand-written C scanner and `flex`.

Lecture slides	Topic	Where, status
48–55	What a lexer does, tokens, attributes, symbol tables	§1, mostly new
56–59	Regular expressions and token patterns	§2, review with new examples
60–64, 73	Thompson (Dragon-book version), NFA simulation, cost	§3, new
65–72	Many tokens at once: longest match, priority, catch-all	§4, extends Ch. 3
74–82	DFA scanning, combined DFA, speed vs. memory, Hopcroft	§5, review plus big example
83	Fortran, PL/I and Python: when lexing is not regular	§6, new
84–90	An ad-hoc scanner in C	§7, new
91–99	<code>flex</code>	§8, new

Every `flex` specification, every C scanner and every trace in this chapter was actually run (flex 2.6.4, a C compiler, and a small Python NFA simulator). The outputs printed are the real outputs, not reconstructions. Section 9 ends with a map from each problem of Problem Set 4 to the sections that prepare it.

1 What a Lexer Is For

1.1 Motivation: the parser should never see a space

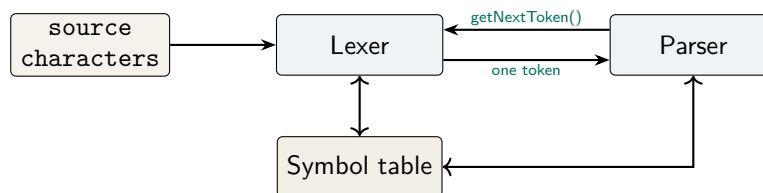
Think about the parser, the phase after the lexer. It wants to reason in terms like “a while-statement is the keyword `while`, an opening parenthesis, an expression, a closing parenthesis, and a statement.” It does not want to know that `while` is five characters long, or that any number of spaces, tabs and newlines may sit between two tokens. Nor does it want to know that a comment may appear almost anywhere. The **lexer** (also called the **scanner**) is the layer that absorbs all of this, so that the parser sees a clean sequence of meaningful symbols.

Mechanism: The four jobs of a lexer

1. **Cut** the character stream into meaningful sequences called *lexemes*.
2. **Label** each lexeme with a *token*, which is what the parser receives.
3. **Record** identifiers (and often numbers) in the *symbol tables*, so that later phases can attach information to them.
4. **Discard** what carries no meaning for the parser: blanks, tabs, newlines and comments.

In one sentence: the lexer is the interface between the raw source program and the parser.

The parser drives the conversation. It does not receive the whole token list at once. Instead it calls the lexer each time it needs one more token (`getNextToken()` in the textbook, `yylex()` in `flex`, `ScanOneToken()` in the C scanner of Section 7).



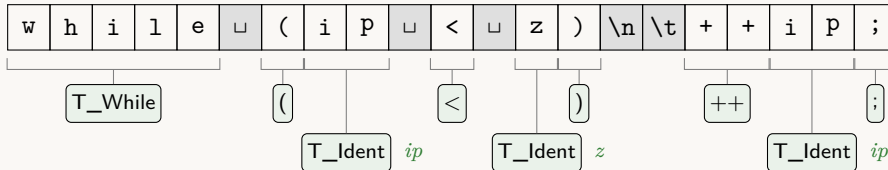
Worked example: From characters to tokens

The two-line C fragment

Key insight. Producing tokens on demand means the lexer never has to store the whole token list. It also lets the parser and lexer cooperate, which Section 6 will need.

```
while (ip < z)
  ++ip;
```

is, for the lexer, one flat stream of characters. Below, the grey cells are characters it discards:



The parser receives nine tokens:

`T_While ( <T_Ident, ip> <T_Ident, z> ) ++ <T_Ident, ip> ;`

Three things already happened that are not obvious.

- **Blanks vanished.** The space, the newline and the tab separate tokens but never reach the parser.
- **Only some tokens carry data.** A `T_Ident` must say *which* identifier it is, so it carries `ip` or `z`. A parenthesis needs nothing more than its name.
- **The lexer decided where each token ends.** It did not stop at `whi` or split `++` into two `+`. Choosing the cut is the central algorithmic problem of this chapter (Section 4).

1.2 Why separate lexical analysis from parsing?

One could write a single grammar that works directly on characters. Nobody does, for three reasons.

Trade-offs: Three reasons for a separate lexer

- **Simplicity of design.** Without a lexer, every grammar rule would have to allow blanks and comments between any two symbols, for example

$$whileStmt \rightarrow \text{while } WS^* (WS^* \text{ expr } WS^*) WS^* stmt.$$

With a lexer the rule is $whileStmt \rightarrow \text{WHILE } (\text{expr}) stmt$. Each half also gets the right tool: regular expressions and finite automata for the lexer, context-free grammars for the parser.

- **Efficiency.** The lexer is the only phase that touches every single character, and there are many times more characters than tokens. That makes it the hottest loop of a classic front end, which justifies specialised techniques: DFA tables, reading the input in large blocks, and one table lookup per character instead of a general parsing algorithm.
- **Portability.** Only the scanner deals with the outside world: character encodings (ASCII, UTF-8), line endings (`\n` versus `\r\n`) and file

reading. To port the compiler, you change the scanner and nothing else.

1.3 Tokens, patterns and lexemes

These three words are used loosely in conversation, but they name three different things.

Concept: Three words, three objects

- A **token** is a pair $\langle \text{name}, \text{attribute} \rangle$. The *name* is what the parser sees: it is a terminal symbol of the grammar. The *attribute* carries whatever extra information later phases need, and it may be multi-valued (for example a symbol-table pointer plus a line and column). Examples: $\langle \text{Ident}, \text{ip} \rangle$, $\langle \text{Operator}, < \rangle$, $\langle \text{"}, \text{NIL} \rangle$.
- A **pattern** describes the set of character strings that can form a lexeme of a token. Examples: “a letter followed by letters and digits”; the finite set $\{<, >, \leq, \geq, ==\}$; the single string “)”.
- A **lexeme** is an actual sequence of characters in the source that matches the pattern of a token. Examples: `ip`, `<` and `)` in `while (ip < z)`.

An analogy: the pattern is a type, the lexeme is a value of that type found in the text, and the token is the report the lexer sends to the parser (the type’s name plus the details that matter).

Key insight. The split is also a split in computational power. The lexer handles the regular part of the syntax; the parser handles the nested, context-free part. Section 6 shows what happens when a language’s lexical structure is *not* regular.

Token name	Pattern	Sample lexemes	Attribute
WHILE	the string <code>while</code>	<code>while</code>	none
IDENT	letter (letter digit)*	<code>ip</code> , <code>z</code> , <code>count1</code>	pointer to the symbol-table entry
RELOP	<code>< <= = <> > >=</code>	<code><</code> , <code>>=</code>	which operator: LT, LE, ...
NUMBER	digits, optional fraction and exponent	<code>60</code> , <code>3.14E-2</code>	the value, or a pointer to a constant table
LPAREN	the string <code>(</code>	<code>(</code>	none

Warning / Gotcha: Where do names like T_Ident come from?

Token names are not built into any tool or language. The compiler writer invents them, as the members of an `enum` or as `#define` constants shared by the lexer and the parser. `T_Ident`, `IDENT`, `ID` and `T_IDENTIFIER` are the same idea spelled four ways. In the C scanner of Section 7 they are plain integers (`#define T_IDENTIFIER 268`). With `flex` and the parser generator `bison`, you declare `%token ID` in the grammar file, and `bison` writes the numbers into a header that both sides include.

1.4 Designing a lexer: three steps

Defining a lexical analyser always takes the same three steps.

1. Define the **set of tokens**.
2. Define a **pattern** for each token, that is, the set of lexemes belonging to it.
3. Define an **algorithm** that cuts the source program into lexemes and outputs the tokens.

Steps 1 and 2 depend on the language; together they are the *lexical specification*. Step 3 is the same for every language, which is exactly why it can be automated (Chapter 3, and **flex** in Section 8).

1.5 Choosing the tokens

The choice depends on the language, but almost every programming language uses the same classes:

- one token for each **keyword** (**if**, **while**, ...);
- one token for each **punctuation** symbol (parentheses, comma, semicolon, ...);
- one token for all **identifiers**;
- several tokens for the **operators**;
- one or more tokens for the **constants** (numbers, string literals).

How do you decide whether two kinds of lexeme deserve two tokens or one token with an attribute? Ask what the parser does with them.

Technique: One token or two?

Give two lexemes different token names exactly when the grammar must treat them differently.

- The six relational operators all appear in the same grammar position, *expr* RELOP *expr*. One token RELOP with an attribute (LT, LE, ...) keeps the grammar at one rule instead of six. Code generation reads the attribute later.
- + and * must be different tokens, because the grammar itself encodes that * binds tighter than +. The parser has to tell them apart.
- Each keyword gets its own token, since each one starts a different construct.

Attributes encode the lexeme when the parser or later phases need it. An identifier carries a pointer to its symbol-table entry, and a constant carries its value. Keywords and punctuation need no attribute, because the token name already says everything.

Concept: What “update the symbol table” means

A **symbol table** is a dictionary from names to information about them. Later phases fill in the type, the scope and the memory location. The

lexer is the first phase to see a name. It inserts the name if it is absent and puts a pointer (or index) to the entry in the token's attribute. Every later occurrence of the same name gets the *same* pointer, so later phases compare names by comparing pointers, never strings. For `x = x + 1` the lexer produces

$$\langle \text{ID}, \bullet \rangle \langle \text{EQ} \rangle \langle \text{ID}, \bullet \rangle \langle + \rangle \langle \text{NUM}, 1 \rangle$$

where both $\bullet$ point to the single entry for `x`. The `flex` example in Section 8 prints these indices, and you will see `x1` receive index 0 both times it appears.

In the real world. Modern compilers often let the lexer only *intern* the string (store one copy and hand out a pointer). The parser and semantic analysis then create the real, scoped symbol entries, because the lexer does not know about scopes.

2 Patterns: Regular Expressions Revisited

2.1 Why regular expressions

A pattern defines the set of lexemes of a token, and a set of strings is a **language**. So a pattern *is* a language. Patterns are normally written as **regular expressions**, which define the **regular languages**. Regular expressions are expressive enough for almost every token, and they compile into fast scanners (finite automata), as Chapter 3 showed.

Concept: Reminder: languages and regular expressions

An **alphabet** Σ is a finite set of characters, such as $\{a, b\}$. A **string** is a finite sequence of them, such as `aabba`. A **language** is a set of strings, such as $\{a, b, abab, babbba\}$. Regular expressions are built from a character a (language $\{a\}$) and ε (language $\{\varepsilon\}$) by three operations: **concatenation** $R_1 R_2$, **alternation** $R_1 \mid R_2$ (union of the languages) and **Kleene star** R^* (zero or more repetitions). Parentheses only group. Star binds tightest, then concatenation, then alternation. Chapter 2 treats all of this in detail.

Worked example: Reading a regular expression: even numbers

The slide's regular expression for even numbers is

$$(+ \mid - \mid \varepsilon) (0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9)^* (0 \mid 2 \mid 4 \mid 6 \mid 8).$$

Read it as a concatenation of three parts and ask what each part contributes: an optional sign, then any number of digits, then one even digit. To test a string, try to assign each character to one part.

String	In L ?	Why
-24	yes	sign -, middle 2, final 4
0	yes	no sign, empty middle, final 0
0034	yes	leading zeros are allowed by this pattern
7	no	the last digit must be even
+	no	a final even digit is mandatory
ε	no	same reason

2.2 Notational conveniences

Real specifications use shorthands. None of them adds expressive power; each can be expanded into the five basic constructs.

- **Regular definitions** give names to subexpressions:
 $letter \rightarrow A \mid B \mid \dots \mid Z \mid a \mid b \mid \dots \mid z$, $digit \rightarrow 0 \mid 1 \mid \dots \mid 9$,
 $id \rightarrow letter (letter \mid digit)^*$.
- **One or more:** $r^+ = rr^*$. **Zero or one:** $r? = r \mid \varepsilon$.
- **Character classes:** $[abc] = a \mid b \mid c$, $[a-z] = a \mid b \mid \dots \mid z$, $[0-9] = 0 \mid 1 \mid \dots \mid 9$.

Warning / Gotcha: Regular definitions are macros, never recursive

A name may only use names defined *before* it. Then every name can be substituted away, and the result is an ordinary regular expression. A recursive definition such as $expr \rightarrow (expr) \mid id$ describes balanced parentheses, which is not a regular language. That needs a grammar. This is exactly the line between the lexer's job and the parser's job.

Worked example: The slide's token patterns, dissected

- **Identifiers** $[a-zA-Z_][a-zA-Z_0-9]^*$. The first character may not be a digit. That keeps 123abc from being an identifier, and it lets a scanner decide from the *first* character alone whether it is reading a number or a name. The hand-written scanner of Section 7 relies on exactly this.
- **Integers** $[+-]?[0-9]^+$: an optional sign, then at least one digit.
- **Floats**, split into a sign, a mantissa and an exponent:
 $[+-]?((([0-9]+(\.[0-9]*)?)|\.[0-9]+)([eE][+-]?[0-9]^+)?)$
 - The mantissa is either digits followed by an optional point and optional digits (3, 3., 3.14), or a point followed by at least one digit (.5).
 - The exponent is optional: e or E, an optional sign, at least one digit.
 - It accepts 3.14e-2, 1E10, .5 and 3.. It rejects ., e5 and 1e.
 - Notice that it also accepts plain integers such as 17. Floats and integers therefore *overlap*, and the scanner needs a rule to decide (Sections 4 and 5).
- **String constants** $"([a-zA-Z0-9]|\backslash[a-zA-Z])^*"$: a double quote,

then letters, digits or backslash-letter escapes, then a closing quote. This is a teaching simplification, because it forbids spaces inside strings. A realistic version is `\"([^\\"\\n]|\\.)*\"`: any character except a quote, a backslash or a newline, or a backslash followed by any character. On the input `"a\"b"` it matches `a`, then the escape `\`, then `b`, then the closing quote, which is one string token (checked with `flex`).

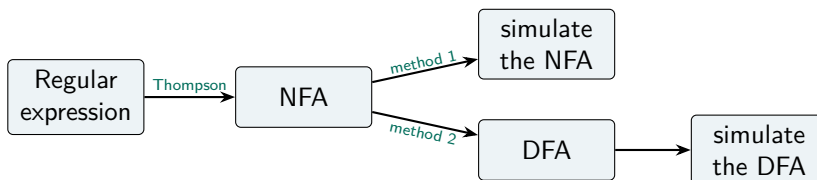
Warning / Gotcha: A sign inside the number pattern is a trap

Take the rules `ID, INT = [+−]?[0−9]+` and `MINUS = −`, and scan `x−1`. At the `−`, the rule `MINUS` matches one character, but `INT` matches the two characters `−1`, and the longer match wins. The lexer outputs $\langle \text{ID}, x \rangle \langle \text{INT}, -1 \rangle$, two operands in a row, and the parser reports a syntax error. Real languages keep the sign *out* of numeric literals and treat it as unary minus in the grammar. The slide's patterns, and the combined DFA of Section 5, are illustrations and keep the sign.

3 Running an NFA Directly

3.1 Motivation: two ways to execute a pattern

Regular expressions, NFAs and DFAs describe exactly the same languages, and finite automata are easy to turn into programs. That leaves two ways to execute a token pattern:



Chapter 3 concentrated on method 2. This section develops method 1, and it is worth understanding for its own sake. A DFA is nothing but a precomputed table of NFA-simulation steps, so once you understand the simulation you understand the DFA as well.

3.2 The NFA, formally

Concept: Nondeterministic finite automaton

Definition 3.1 (NFA). An NFA is a five-tuple $M = (Q, \Sigma, \Delta, s_0, F)$ where Q is a finite set of states, Σ an alphabet, $s_0 \in Q$ the initial state, $F \subseteq Q$ the accepting states, and

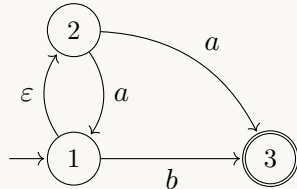
$$\Delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times Q$$

the **transition relation**. Each triple $(q, x, q') \in \Delta$ is an edge $q \xrightarrow{x} q'$. A

string is accepted if *some* path from s_0 to a state of F spells it, ignoring ε labels.

Because Δ is a relation and not a function, a transition-table entry is a *set* of states, possibly empty.

Worked example: The slide's NFA, and what it accepts



State	a	b	ε
1	$\emptyset$	$\{3\}$	$\{2\}$
2	$\{1, 3\}$	$\emptyset$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

The nondeterminism sits in the entry $(2, a) = \{1, 3\}$: on an a in state 2 the machine may go back to 1 or finish in 3.

Which strings does it accept? Every trip around the cycle $1 \xrightarrow{\varepsilon} 2 \xrightarrow{a} 1$ reads one a . The machine leaves the cycle either by $1 \xrightarrow{b} 3$ or by $2 \xrightarrow{a} 3$. So

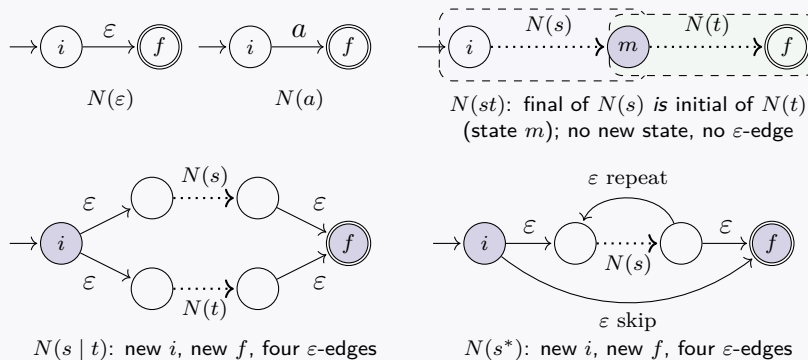
$$L = \{a^n b : n \geq 0\} \cup \{a^n : n \geq 1\} = L(a^* b \mid a^+).$$

For aab the accepting path is $1 \xrightarrow{\varepsilon} 2 \xrightarrow{a} 1 \xrightarrow{\varepsilon} 2 \xrightarrow{a} 1 \xrightarrow{b} 3$. For ba no path exists, because state 3 has no outgoing edge.

3.3 Thompson's construction, Dragon-book convention

The lecture's Thompson construction (slide 62, taken from the Dragon book) builds $N(r)$ by induction on r , exactly as in Chapter 3. There is one difference, in concatenation. Chapter 3 joined $N(s)$ and $N(t)$ with an ε -edge. The Dragon book instead **merges** the final state of $N(s)$ with the initial state of $N(t)$, so concatenation adds no state and no edge. Problem Set 4 uses this merged convention, and so does the rest of this chapter.

Mechanism: Thompson's rules, merged-concatenation version



Dotted arrows stand for a whole sub-automaton, from its initial state to

its final state. New states are shaded.

The slide lists three properties of $N(r)$. They are not decoration: each one is used later.

Proposition 3.2 (Properties of Thompson NFAs). *For every regular expression r :*

- (P1) $N(r)$ has at most twice as many states as r has operators and operands.
- (P2) $N(r)$ has exactly one initial state and one accepting state. No edge enters the initial state, and no edge leaves the accepting state.
- (P3) Every non-accepting state has either exactly one outgoing edge (on a symbol or on ε), or exactly two outgoing edges, both on ε .

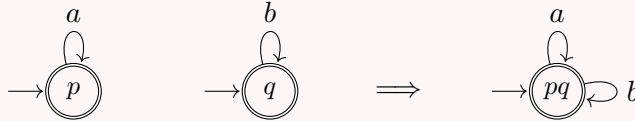
Proof. (P2) and (P3) follow by induction on r . The base cases are visible in the figure. For $s \mid t$ and s^* , the new initial state has two outgoing ε -edges and no incoming edge, and the new final state has no outgoing edge. The old final state of $N(s)$ gains one outgoing ε -edge in $s \mid t$, and two (to f and back to $N(s)$'s start) in s^* , which is allowed by (P3). The back edge of s^* enters $N(s)$'s old initial state, but that state is no longer the initial state of the whole automaton, so (P2) still holds.

For st , the merged state m was the final state of $N(s)$, with no outgoing edges by (P2), and the initial state of $N(t)$, with no incoming edges by (P2). After merging, m has $N(s)$'s incoming edges and $N(t)$'s outgoing edges. Those outgoing edges satisfied (P3) in $N(t)$, so they still do.

For (P1), see the exact count in Proposition 3.3. □

Warning / Gotcha: What goes wrong without (P2)

Suppose you build $N(a^*)$ “lazily” as one state that is both initial and accepting, with a loop on a , and $N(b^*)$ the same way with a loop on b . Neither satisfies (P2). Merging them for a^*b^* produces one state with both loops:



This machine accepts ba , which is not in $L(a^*b^*)$. Thompson's rules spend two extra states per star precisely to avoid this.

Key insight. (P2) is what makes merging safe. Because nothing leaves $N(s)$'s final state and nothing enters $N(t)$'s initial state, gluing them cannot create a path that skips back and forth between the two halves.

Proposition 3.3 (Exact state and edge counts, merged convention). *Let r contain n_{leaf} leaves (symbols or ε), $n_{|}$ alternations, n_{*} stars and $n_{\cdot}$ concatenations. Then*

$$|Q| = 2(n_{leaf} + n_{|} + n_{*}) - n_{\cdot}, \quad |E| = n_{leaf} + 4(n_{|} + n_{*}).$$

Since there are $n_l + n_* + n_c$ operators and n_{leaf} operands,

$$|Q| = 2(\#operators + \#operands) - 3n_c \leq 2(\#operators + \#operands),$$

with equality exactly when r contains no concatenation.

Proof. Each leaf creates 2 states and 1 edge. Each alternation and each star creates 2 states and 4 edges. Each concatenation merges two existing states into one, so the state count drops by exactly 1 and no edge is created. Sum over the syntax tree. $\square$

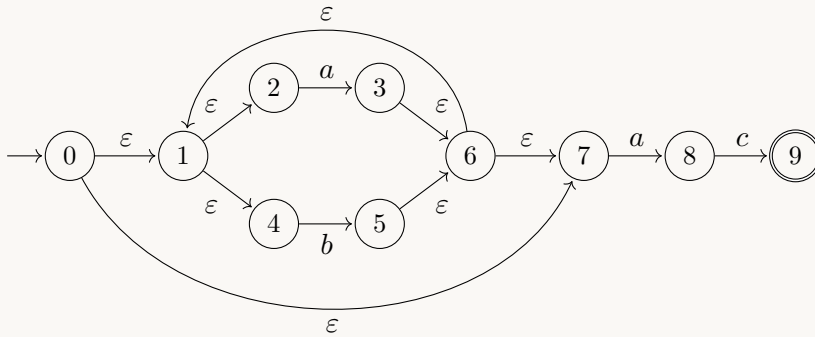
Key insight. Parentheses are not operators: they only group and create nothing. Count the operators in the syntax tree, not the symbols on the page.

Worked example: Thompson for $(a \mid b)^*ac$, step by step

Parse first. $(a \mid b)^*ac = ((a \mid b)^* \cdot a) \cdot c$. There are 4 leaves (a, b, a, c) , 1 alternation, 1 star and 2 concatenations.

Build inside out.

1. $N(a)$: $2 \xrightarrow{a} 3$. $N(b)$: $4 \xrightarrow{b} 5$.
2. $N(a \mid b)$: new states 1 and 6; edges $1 \xrightarrow{\varepsilon} 2$, $1 \xrightarrow{\varepsilon} 4$, $3 \xrightarrow{\varepsilon} 6$, $5 \xrightarrow{\varepsilon} 6$.
3. $N((a \mid b)^*)$: new states 0 and 7; edges $0 \xrightarrow{\varepsilon} 1$, $6 \xrightarrow{\varepsilon} 7$, $7 \xrightarrow{\varepsilon} 0$ (repeat), $0 \xrightarrow{\varepsilon} 7$ (skip).
4. Concatenate with $N(a)$: its initial state is merged into 7, so the new edge is $7 \xrightarrow{a} 8$.
5. Concatenate with $N(c)$: its initial state is merged into 8, so the new edge is $8 \xrightarrow{c} 9$. State 9 is final.



Check the counts. By Proposition 3.3, $|Q| = 2(4 + 1 + 1) - 2 = 10$ (states 0 to 9) and $|E| = 4 + 4 \cdot 2 = 12$. Count the edges in the figure: there are twelve. There are 8 operators and operands, so the bound (P1) is $16 \geq 10$.

Check (P3). States 0, 1 and 6 have two outgoing ε -edges. States 2, 3, 4, 5, 7 and 8 have exactly one outgoing edge. State 9 is final and has none.

Warning / Gotcha: The slide's example NFA is *not* built by these rules

Slide 63 draws an 8-state NFA for $(a \mid b)^*ac$ (states 1 to 8, taken from Mogensen's textbook) right next to the list of Thompson properties. That NFA comes from a different, more economical construction. Its initial

state 1 has an incoming edge $8 \xrightarrow{\varepsilon} 1$, which violates (P2). Both NFAs are correct for the language. Only the Dragon-book rules guarantee the three properties and the exact count above, so use them when a question says “Thompson’s construction”.

3.4 Simulating an NFA

The difficulty with an NFA is that at each step it may have several choices. The solution is not to choose. Keep the *set* of all states the NFA could be in, and update that set one character at a time.

Mechanism: The two operations

- $move(S, c) = \{q' : q \in S, (q, c, q') \in \Delta\}$, the states reachable from some state of S by one edge labelled c .
- ε -closure(T) is the set of states reachable from some state of T by zero or more ε -edges. It always contains T itself.

Mechanism: NFA simulation (slide 64, Dragon book)

Algorithm 1: Does the NFA accept the input?

```

1  $S \leftarrow \varepsilon$ -closure( $\{s_0\}$ );
2  $c \leftarrow nextChar()$ ;
3 while  $c \neq eof$  do
4    $S \leftarrow \varepsilon$ -closure( $move(S, c)$ );
5    $c \leftarrow nextChar()$ ;
6 if  $S \cap F \neq \emptyset$  then return “yes”;
7 else return “no”;

```

The closure is a plain graph search over ε -edges:

Mechanism: Computing ε -closure(T)

Algorithm 2: ε -closure with a stack

```

1 push every state of  $T$  onto a stack;  $C \leftarrow T$ ;
2 while the stack is not empty do
3   pop  $t$ ;
4   foreach edge  $t \xrightarrow{\varepsilon} u$  do
5     if  $u \notin C$  then
6        $C \leftarrow C \cup \{u\}$ ; push  $u$ ;
7 return  $C$ ;

```

A state enters C at most once, so each state is pushed and popped at most once.

Proposition 3.4 (The simulation is correct). *After the loop has consumed a prefix w of the input, S is exactly the set of states q such that some path from*

s_0 to q spells w . Hence the algorithm answers “yes” if and only if the NFA accepts the input.

Proof. By induction on $|w|$. For $w = \varepsilon$, the paths spelling ε use only ε -edges, and those reach exactly $\varepsilon\text{-closure}(\{s_0\})$. For $w = vc$, a path spelling vc is a path spelling v to some q , then one c -edge, then ε -edges. By the induction hypothesis the possible q form the old S , so the possible endpoints are $\varepsilon\text{-closure}(\text{move}(S, c))$, the new S . $\square$

Worked example: Simulating the Thompson NFA of $(a \mid b)^*ac$

Read	move	S after closure
(start)		$\{0, 1, 2, 4, 7\}$
b	$\{5\}$	$\{1, 2, 4, 5, 6, 7\}$
a	$\{3, 8\}$	$\{1, 2, 3, 4, 6, 7, 8\}$
c	$\{9\}$	$\{9\}$

At the end $9 \in S$, so **bac** is accepted. On **ab** the sets are $\{0, 1, 2, 4, 7\}$, then $\{1, 2, 3, 4, 6, 7, 8\}$, then $\{1, 2, 4, 5, 6, 7\}$. That last set contains no 9, so **ab** is rejected.

Look at the closure after **b**. From 5 the ε -edges lead to 6, from 6 to 7 and back to 1, and from 1 to 2 and 4. One character can therefore pull in a whole cascade of states.

Key insight. NFA simulation is subset construction done lazily: one input character at a time, and only for the sets that this input actually visits.

3.5 What it costs

The slide states that an NFA with $|Q|$ states needs $O(|S| |Q|^2)$ time on a string of length $|S|$. Here is where the square comes from, and why Thompson NFAs do better.

Proposition 3.5 (Cost of NFA simulation). *Represent S as a list of states plus a Boolean array $\text{inS}[q]$.*

1. For an arbitrary NFA, one step $S \leftarrow \varepsilon\text{-closure}(\text{move}(S, c))$ costs $O(|Q|^2)$, so the whole input costs $O(|S| |Q|^2)$.
2. For a Thompson NFA, one step costs $O(|Q|)$, so the whole input costs $O(|S| |Q|)$.
3. For a DFA, one step is one table lookup, so the whole input costs $O(|S|)$, independent of $|Q|$.

Proof. One step does two things. move scans the outgoing edges of each state of S . The closure pushes each state at most once (the array inS makes the membership test $O(1)$) and scans the outgoing ε -edges of each popped state. So one step costs $O(|Q| + \text{number of edges scanned})$.

In an arbitrary NFA, a state may have up to $|Q|$ successors on the same symbol and up to $|Q|$ on ε . Up to $|Q|$ states may be in S , so a step may scan $\Theta(|Q|^2)$

edges. **That is where the square comes from: the number of edges, not the number of states.**

In a Thompson NFA, (P3) says every state has at most two outgoing edges, so a step scans at most $2|Q|$ edges and costs $O(|Q|)$. Clearing *inS* between steps must be done by walking the list, not the whole array, to stay within this bound.

A DFA has exactly one current state and one successor per symbol, so a step is a single lookup in δ . $\square$

Trade-offs: NFA simulation versus DFA, for a pattern of length m

A Thompson NFA has $|Q| \leq 2m$ states.

	Simulate the NFA	Build and run the DFA
Preparation	$O(m)$ (Thompson)	up to $O(2^m)$ (subset construction)
Memory	$O(m)$	up to $O(2^m \cdot \Sigma)$ table entries
Time on input of length $ S $	$O(S \cdot m)$	$O(S)$

A compiler's token specification is fixed and its input is long, so paying the preparation once is worth it: lexers use DFAs. A one-off search with a user-typed pattern is the opposite case, and tools like `grep` use NFA simulation or build DFA states lazily (Section 5.4).

4 From Recognising to Cutting: Many Tokens at Once

4.1 Motivation: the real question is “where does the next token end?”

So far we have one regular expression per token, one NFA per token, and a way to simulate an NFA. A simulated NFA answers the question “is this whole string in the language?”. A lexer faces a different situation. It has *many* patterns and one long, unsegmented stream, and it must decide where the next token ends and which token it is. The slide offers two ways to combine the pieces.

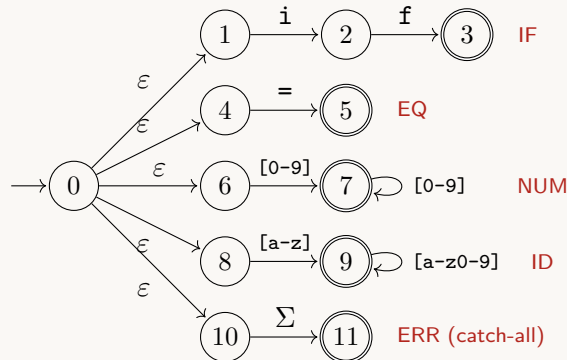
- **Run the token NFAs separately**, in turn or in parallel, from the current position, and report the token of the one that reaches an accepting state.
- **Merge all the NFAs into one.** Add a new start state with an ε -edge to the start of each token NFA, and label each accepting state with its token name.

In practice the second way is used (slide 71). One simulation does all the work, and the merged NFA is what we will later turn into a DFA. The first way, read literally (“the first one to reach an accepting state”), would also stop

too early: after reading `i` of `ifu26`, the identifier automaton already accepts. The longest-match principle below fixes this.

Worked example: Four tokens, one merged NFA

Tokens: IF = `if`, EQ = `'='`, NUM = `[0-9]+`, ID = `[a-z][a-z0-9]*`, plus a catch-all rule ERR on states 10 and 11 that is explained in a moment. The numbering follows the slides.



On `x = 60`, with blanks skipped, the lexer outputs $\langle \text{ID}, x \rangle \langle \text{EQ} \rangle \langle \text{NUM}, 60 \rangle$.

4.2 Ambiguity 1: where does a token end?

Now scan `ifu26 = 60`. Several cuts are consistent with the patterns:

$\langle \text{IF} \rangle \langle \text{ID}, u26 \rangle \langle \text{EQ} \rangle \langle \text{NUM}, 60 \rangle$
 $\langle \text{ID}, ifu26 \rangle \langle \text{EQ} \rangle \langle \text{NUM}, 60 \rangle$
 $\langle \text{ID}, ifu \rangle \langle \text{NUM}, 26 \rangle \langle \text{EQ} \rangle \langle \text{NUM}, 6 \rangle \langle \text{NUM}, 0 \rangle$
 ...

A lexer must produce exactly one of them, and every programmer expects the second.

Concept: Principle of the longest matching prefix

At each position, choose the **longest prefix** of the remaining input that matches *any* token pattern. Then restart right after it. This is also called **maximal munch**.

Longest match turns `ifu26` into one identifier, `<=` into one operator rather than `<` then `=`, and `++` into one token. It also makes the result unique, because at each position there is only one longest length.

Mechanism: Implementing longest match on the merged NFA

Run the merged NFA from the current position, and remember the **last match**: the token label and end position from the most recent time S contained an accepting state. When S becomes empty (every

automaton is stuck), or at end of input, report the last match and restart just after it. Every character read *after* the last match must be pushed back into the input, because it belongs to the next token.

Algorithm 3: Scan one token from position p

```

1  $S \leftarrow \varepsilon\text{-closure}(\{s_0\}); \quad i \leftarrow p; \quad \text{last} \leftarrow \text{none};$ 
2 while  $i < n$  do
3    $S \leftarrow \varepsilon\text{-closure}(\text{move}(S, \text{input}[i])); \quad // \text{ one read}$ 
4   if  $S = \emptyset$  then break    // stuck: this read is wasted;
5    $i \leftarrow i + 1;$ 
6   if  $S \cap F \neq \emptyset$  then  $\text{last} \leftarrow (\text{best label in } S \cap F, i);$ 
7 if  $\text{last} = \text{none}$  then lexical error;
8 emit  $\text{last.label}$  with lexeme  $\text{input}[p.. \text{last.end} - 1];$ 
9  $p \leftarrow \text{last.end}; \quad // \text{ everything read after it is pushed back}$ 

```

Worked example: Tracing ifu26=60 on the merged NFA

Written without blanks, so that the step the slide mentions (“= is read and then re-inserted”) appears exactly. Priority order: IF, EQ, NUM, ID, ERR.

Read	S after the step	Accepting labels	Last match
<i>Token 1, start</i> {0, 1, 4, 6, 8, 10}			
i	{2, 9, 11}	ID, ERR $\Rightarrow$ ID	(ID, 1 char)
f	{3, 9}	IF, ID $\Rightarrow$ IF	(IF, 2)
u	{9}	ID	(ID, 3)
2	{9}	ID	(ID, 4)
6	{9}	ID	(ID, 5)
=	$\emptyset$	stuck	emit ID “ifu26”, push back =
<i>Token 2</i>			
=	{5, 11}	EQ, ERR $\Rightarrow$ EQ	(EQ, 1)
6	$\emptyset$	stuck	emit EQ , push back 6
<i>Token 3</i>			
6	{7, 11}	NUM, ERR $\Rightarrow$ NUM	(NUM, 1)
0	{7}	NUM	(NUM, 2)
EOF			emit NUM “60”

Ten reads for eight characters, since = and 6 were each read twice. Watch the row for **f**: after two characters the best label is IF, but the scanner does *not* stop. It keeps reading, and the longer identifier match replaces IF. Priority only decides between matches of the *same* length.

4.3 Ambiguity 2: one lexeme, two tokens

The lexeme **if** is accepted both by the IF automaton and by the ID automaton, with the same length. Longest match cannot decide. The slide gives two solutions:

- **Forbid it:** report an error if the language definition allows such a conflict.
- **Priority:** let the specification order the tokens, for example keywords before identifiers.

Priority is implemented by labelling: when S contains several accepting states, the label of the highest-priority one wins. In **flex** the priority is simply the order in which the rules are written (Section 8).

Trade-offs: Keywords as patterns, or keywords by table lookup?

- **As patterns** (the merged NFA above, and most **flex** specifications): one rule per keyword, placed before the identifier rule. It is simple and declarative, but each keyword adds states to the automaton.
- **By lookup** (the C scanner of Section 7, and many production compilers): a single identifier rule, then a hash-table lookup of the lexeme in a keyword table. The automaton stays small and the keyword list is easy to change. It costs one lookup per identifier.

Both give the same tokens, because both implement “same lexeme, keyword wins”.

4.4 Ambiguity 3: nothing matches

What if no accepting state is reachable from the current position, for example on $\$$ when no rule mentions $\$$? Then the algorithm above stops with *last* = none, and the scanner has no token to emit. The slide’s answer is to add a **catch-all rule**: one more automaton, $10 \xrightarrow{\Sigma} 11$, that accepts any single character, has the **lowest** priority, and reports an error.

Proposition 4.1 (The catch-all never steals a real token). *With the catch-all rule placed last, it is chosen at position p if and only if no other rule matches any non-empty prefix of the input at p .*

Proof. The catch-all matches exactly one character. If another rule matches a prefix of length ≥ 2 , longest match prefers that rule. If another rule matches a prefix of length exactly 1, there is a tie, and priority prefers the other rule because the catch-all is last. Only if no other rule matches anything does the catch-all’s one-character match remain. $\square$

After reporting the error, the scanner skips that one character and continues. This is the simplest form of error recovery, called *panic mode*.

Summary: Lexical analysis with an NFA (slide 72)

1. Construct an NFA for each token’s regular expression.
2. Merge them into one automaton with a new start state; label the accepting states with token names.
3. Scan the input, keeping track of the last known match.
4. Break ties by choosing the higher-priority token.
5. Have a catch-all rule to handle errors.

5 Scanning with a DFA

5.1 From relation to function

A DFA is an NFA with two restrictions: no ε -edges, and exactly one transition for every (state, letter) pair. The transition relation becomes a **function** $\delta : Q \times \Sigma \rightarrow Q$. Missing transitions go to a non-accepting dead state, as in Chapter 3. Simulation (slide 77) then needs no sets at all:

Mechanism: DFA simulation

```
1  $s \leftarrow s_0$ ;  $c \leftarrow nextChar()$ ;
2 while  $c \neq eof$  do  $s \leftarrow \delta(s, c)$ ;  $c \leftarrow nextChar()$ ;
3 if  $s \in F$  then return “yes”;
4 else return “no”;
```

Time $O(|S|)$, independent of the number of states.

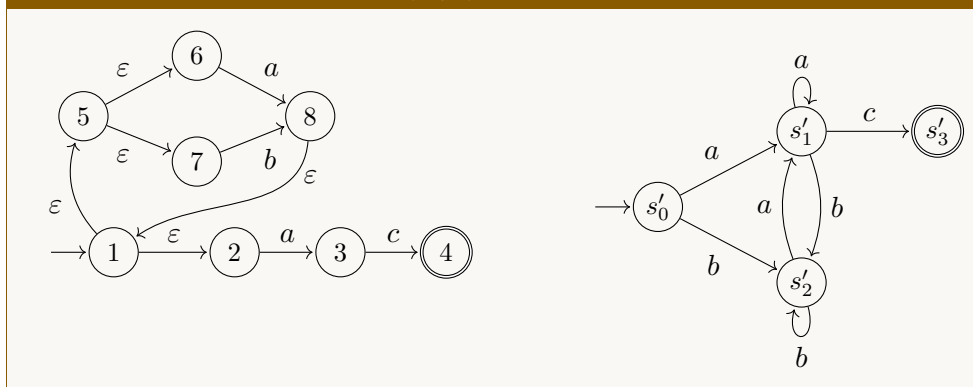
5.2 Subset construction, in the lecture’s notation

The subset construction (Chapter 3, in full) mimics the NFA simulation *in advance*, for every possible input:

- each DFA state is a **set** of NFA states, and the first one is ε -closure(s_0);
- the transitions are $\delta(S, c) = \varepsilon$ -closure(move(S, c)), the very same step as in the simulation;
- a DFA state is accepting if it contains at least one accepting NFA state.

Only the sets reachable from the start are ever built. Use a worklist: start with the initial set, and for each set taken from the list compute its successor on every letter, adding any set not seen before.

Worked example: Slide 76: $(a \mid b)^*ac$, from Mogensen’s 8-state NFA



DFA state	NFA set	on a	on b	on c
s'_0 (start)	$\varepsilon\text{-cl}(1) = \{1, 2, 5, 6, 7\}$	$\{3, 8\} \rightarrow s'_1$	$\{8\} \rightarrow s'_2$	$\emptyset$
s'_1	$\{1, 2, 3, 5, 6, 7, 8\}$	$\{3, 8\} \rightarrow s'_1$	$\{8\} \rightarrow s'_2$	$\{4\} \rightarrow s'_3$
s'_2	$\{1, 2, 5, 6, 7, 8\}$	$\{3, 8\} \rightarrow s'_1$	$\{8\} \rightarrow s'_2$	$\emptyset$
s'_3 (accepting)	$\{4\}$	$\emptyset$	$\emptyset$	$\emptyset$

The “on” columns show *move* before the closure, then the resulting DFA state. For example, $\text{move}(s'_0, a) = \{3, 8\}$, and the closure adds 1 (from 8), then 2 and 5, then 6 and 7, giving $\{1, 2, 3, 5, 6, 7, 8\} = s'_1$.

Two remarks.

- Run the same construction on the 10-state Thompson NFA of Section 3.3 and you get four sets again: $\{0, 1, 2, 4, 7\}$, $\{1, 2, 3, 4, 6, 7, 8\}$, $\{1, 2, 4, 5, 6, 7\}$ and $\{9\}$, with exactly the same transitions. Different NFAs, same DFA up to renaming.
- This DFA is not minimal. Rows s'_0 and s'_2 are identical and both non-accepting, so the two states are equivalent and merge. The minimal DFA has three states: “nothing useful yet”, “just read an a ” (s'_1), and “just read ac ” (s'_3).

5.3 The combined DFA for several tokens

For a lexer, the subset construction runs on the *merged* NFA, and the token labels must be carried over (slide 78):

Mechanism: Labelling DFA states

A DFA state that contains accepting NFA states of several tokens gets the label of the highest-priority one among them. Scanning then works exactly as with the NFA: run the DFA, remember the last accepting state and its position, and when stuck, emit, push back and restart.

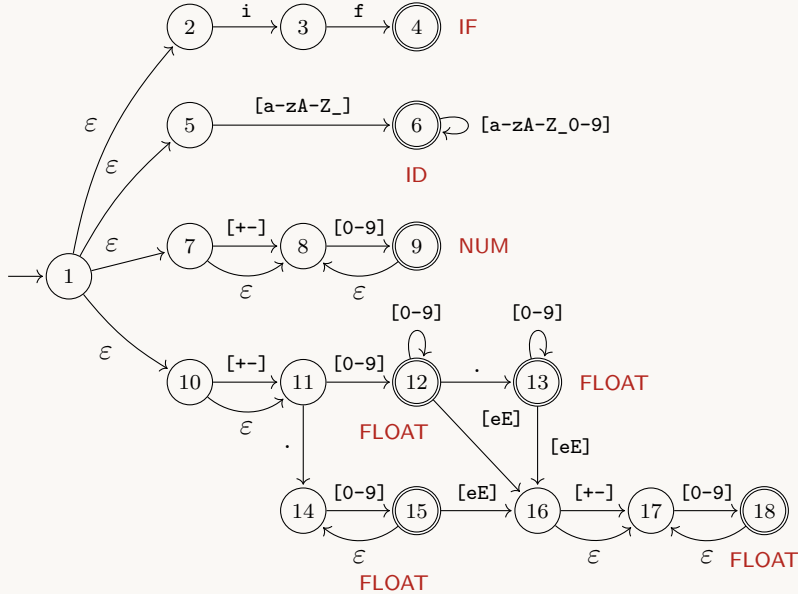
The lecture’s big example (slides 79–80, from Mogensen) has four tokens, in this priority order:

```

IF      if
ID      [a-zA-Z_][a-zA-Z_0-9]*
NUM     [+]?[0-9]+
FLOAT   [+]?((([0-9]+(\.[0-9]*)?)|\.[0-9]+)([eE][+-]?[0-9]+)?)

```

Worked example: The combined NFA (slide 79)



NUM is the chain 7–9, which reads an optional sign and then digits.
 FLOAT is the rest:

- 10–12 read a sign and the integer part;
- 12–13 read a point and the fraction;
- 11–15 read the “.5” form;
- 16–18 read the exponent.

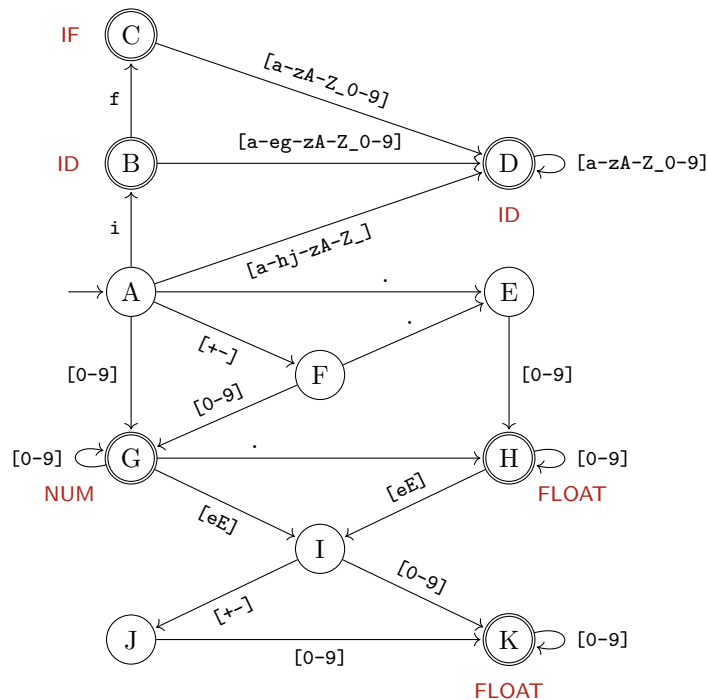
State 12 is accepting because the FLOAT pattern also matches plain integers, as noted in Section 2.

Worked example: Its subset construction

Running the worklist on the merged NFA gives the following reachable sets. The letters are the slide’s names. “Other letters” means the letters not listed separately.

State	NFA set	Label	Transitions
A	{1, 2, 5, 7, 8, 10, 11}	—	i→B, other letters→D, [+–]→F, [0–9]→G, .→E
B	{3, 6}	ID	f→C, other letters/digits→D
C	{4, 6}	IF (over ID)	letters/digits→D
D	{6}	ID	letters/digits→D
E	{14}	—	[0–9]→H
F	{8, 11}	—	[0–9]→G, .→E
G	{8, 9, 12}	NUM (over FLOAT)	[0–9]→G, .→H', [eE]→I
H	{14, 15}	FLOAT	[0–9]→H, [eE]→I
H'	{13}	FLOAT	[0–9]→H', [eE]→I
I	{16, 17}	—	[+–]→J, [0–9]→K
J	{17}	—	[0–9]→K
K	{17, 18}	FLOAT	[0–9]→K

That is twelve states. The slide shows eleven, because H and H' have the same label and identical transitions. They are equivalent, and the slide has already merged them, which is a small minimisation step. Two labelling decisions matter here: C contains accepting states of IF and ID, and G contains accepting states of NUM and FLOAT. In both cases the earlier token wins.



The combined DFA of slide 80. Missing transitions go to the dead state.

Worked example: Slide 80's exercise: lex if 17 and 3e-y

Assume, as the slide implicitly does, a rule that skips blanks and a catch-all rule, neither of which is drawn. A read counts every character fetched from the input, including the one on which the DFA gets stuck.

Input if 17.

Reads	Path	Result
i, f, $\square$	A $\rightarrow$ B (ID) $\rightarrow$ C (IF) $\rightarrow$ stuck	emit IF , push back $\square$
$\square$, 1	blank rule, then stuck	skip the blank, push back 1
1, 7	A $\rightarrow$ G (NUM) $\rightarrow$ G (NUM), end of input	emit NUM 17

Seven reads in total. The **if** is IF because C's label is IF, and 17 is NUM, not FLOAT, because G's label is NUM.

Input 3e-y.

Reads	Path	Result
3, e, -, y	A → G (NUM, last match) → I → J → stuck	emit NUM 3 , push back e-y
e, -	A → D (ID) → stuck	emit ID e , push back -
-, y	A → F → stuck, no accepting state	catch-all: error on - , push back y
y	A → D (ID), end of input	emit ID y

Nine reads for four characters. After **3e-** the DFA was hoping for an exponent like **3e-5**. When **y** arrived it had to give back **three** characters (e, - and the stuck y) and fall back to the last accepting state G. This is longest match with real backtracking. Notice also that - alone is not a token of this specification, so only the catch-all saves the scan.

Technique: How far can a scanner push back?

For one token, the pushback is the number of characters read after the last accepting state, including the stuck one. Read off the DFA, the maximum pushback is

$$1 + \ell,$$

where ℓ is the length of the longest path through *non-accepting* states that starts right after an accepting state. In the DFA above, the longest such path is $G \xrightarrow{e} I \xrightarrow{-} J$, two non-accepting states, so the maximum pushback is $2+1 = 3$, reached by **3e-y**. If a *cycle* of non-accepting states is reachable after an accepting state, the pushback is unbounded. That is the warning sign of the quadratic behaviour studied in Chapter 3 and in Section 9.

5.4 Speed versus memory

The DFA is fast, but it can be large. The number of DFA states can grow *exponentially* with the size of the regular expression or NFA.

Worked example: The classic blow-up family

Let $L_k = (a \mid b)^* a (a \mid b)^{k-1}$, the strings whose k -th symbol from the end is a . An NFA needs only $k+1$ states: it guesses where the important a is and then counts $k-1$ more symbols. Any DFA needs at least 2^k states. It must remember the last k symbols, and all 2^k possibilities really are different:

Let $u \neq v$ be two strings of length k . They differ at some position i ; say $u_i = a$ and $v_i = b$. Append b^{i-1} . The string ub^{i-1} has length $k+i-1$, so its k -th symbol from the end is at position i , which is $u_i = a$: it is in L_k . By the same argument vb^{i-1} is not. A DFA that reached the same state after u and after v would give the same answer on both extended strings, which is impossible. So the 2^k strings of length k need 2^k different states.

This is the argument of Chapter 3’s blow-up proposition, written for a instead of 0.

So we must choose between **low-memory, slower NFAs** and **high-memory, fast DFAs**. For lexers the DFA almost always wins. Token patterns are short and overlap little, and on the `count.1` example of Section 8, `flex -v` reports 15 NFA states and only 7 DFA states. The blow-up is a real danger for *search* patterns.

In the real world. `grep` and the RE2 library build DFA states *lazily*: only when the input reaches them, cached, with a cap on the cache size. This is NFA simulation with memoisation, and it gets the best of both.

5.5 Minimisation and Hopcroft’s algorithm

Theorem 5.1 (Myhill–Nerode, uniqueness). *Every regular language has a unique minimal DFA, up to renaming of states.*

Chapter 3 minimised by table filling, marking distinguishable pairs. That method looks at all pairs, which costs $O(n^2)$ for n states. **Hopcroft’s algorithm** reaches $O(n \log n)$ for a fixed alphabet.

Mechanism: Hopcroft’s idea: partition refinement, smaller half first

1. Start with two blocks: F and $Q \setminus F$.
2. A *splitter* is a pair (block C , letter c). It splits a block B into the states whose c -successor lies in C and those whose c -successor does not. Whenever such a split is non-trivial, replace B by its two parts.
3. Keep a worklist of splitters. When a block is split, add only the **smaller** of the two halves as a new splitter.
4. When the worklist is empty, the blocks are the states of the minimal DFA.

Why $n \log n$? A state is charged work only when it lies in a block used as a splitter. Each time that happens again, the block containing it is at most half the size of the previous one, so this can happen at most $\log_2 n$ times per state.

Worked example: Hopcroft on the $(a \mid b)^*ac$ DFA

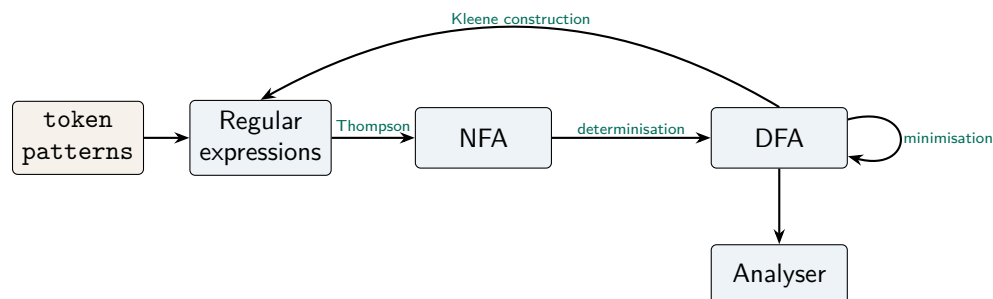
1. Start with the blocks $\{s'_3\}$ and $\{s'_0, s'_1, s'_2\}$.
2. Splitter $(\{s'_3\}, c)$: only s'_1 goes to $\{s'_3\}$ on c , so the big block splits into $\{s'_1\}$ and $\{s'_0, s'_2\}$.
3. Splitters with $\{s'_1\}$: on a , both s'_0 and s'_2 go to s'_1 ; on b and c , neither does. No further split.

The result has three blocks, which is the 3-state minimal DFA found in the remark above.

Warning / Gotcha: Minimisation does not cure blow-up

The 2^k lower bound holds for *every* DFA of L_k , including the minimal one. Minimisation removes redundant states, not necessary ones.

5.6 The whole picture



The arrow back, the **Kleene construction**, turns any DFA into a regular expression. It closes the circle and proves that the three formalisms describe exactly the same languages. One way to perform it is *state elimination*: remove states one at a time, and label the edges with regular expressions instead of letters. When you remove a state q that has a self-loop L , every pair of edges $p \xrightarrow{A} q \xrightarrow{B} r$ becomes a new edge $p \xrightarrow{AL^*B} r$, joined by $|$ with any edge already present.

Worked example: Kleene's construction on the minimal $(a | b)^*ac$ DFA

The states are $P = \{s'_0, s'_2\}$ (start), s'_1 and s'_3 (accepting), with edges $P \xrightarrow{b} P$, $P \xrightarrow{a} s'_1$, $s'_1 \xrightarrow{a} s'_1$, $s'_1 \xrightarrow{b} P$ and $s'_1 \xrightarrow{c} s'_3$. Eliminating s'_1 (loop a) turns $P \rightarrow s'_1 \rightarrow P$ into aa^*b , which joins the loop b to give $b | aa^*b = a^*b$, and turns $P \rightarrow s'_1 \rightarrow s'_3$ into aa^*c . The language is therefore $(a^*b)^*a^+c$. It looks nothing like $(a | b)^*ac$, yet it is the same language: a string of $(a | b)^*a$ always splits into a part that is empty or ends with b , then a block of a 's.

6 When the Lexical Structure Fights Back

6.1 Motivation: the model has assumptions

Everything so far assumes three things. Tokens are separated by blanks or by longest match. Whether a lexeme is a keyword depends only on the lexeme itself. And the lexer needs only finite memory, namely the current DFA state. Slide 83 shows three real languages that break each assumption in turn.

6.2 Fortran: blanks do not matter

In Fortran (up to Fortran 77), blanks are insignificant *everywhere*, even inside names. Compare

Source line	Without blanks	Meaning
D0 5 I = 1,25	D05I=1,25	loop: D0 to label 5, variable I from 1 to 25
D05I = 1.25	D05I=1.25	assignment of 1.25 to the variable D05I

Without blanks, the two lines are identical for six characters. They differ only at the seventh, the comma versus the point. Longest match would read D05I as an identifier in both lines, because the identifier match is longer than the keyword D0. That is wrong for the loop.

The lexer needs **lookahead beyond the end of the token**. It may only decide that D0 is a keyword after checking that a comma follows the = later in the statement. **flex** offers this as **trailing context**: **r/s** matches **r** only if it is followed by **s**, and **s** is not consumed.

```
D0/[A-Z0-9]*=[A-Z0-9]*,    printf("[KEYWORD D0]");
[A-Z][A-Z0-9]*             printf("[ID %s]", yytext);
[0-9]+(\\.[0-9]+)?         printf("[NUM %s]", yytext);
[=,]                       printf("[%s]", yytext);
```

On the blank-free lines this prints (real output):

```
[KEYWORD D0] [NUM 5] [ID I] [=] [NUM 1] [,] [NUM 25]
[ID D05I] [=] [NUM 1.25]
```

In the first line the keyword rule matches 2 characters while the identifier rule matches 4. Why does the keyword win? In **flex**, the length of a trailing-context rule *includes* the trailing context **s** (here the whole D05I=1, part), so the keyword rule has the longest match. Only D0 is consumed, though.

History. A widely retold story, possibly apocryphal, blames a NASA software error on a D0 loop typed with a point instead of a comma. The compiler accepted it silently as an assignment.

6.3 PL/I: keywords are not reserved

In PL/I, *any* keyword may also be used as an identifier. The slide's (legal) line is

```
IF THEN THEN THEN = ELSE; ELSE ELSE = IF
kw   var   kw   var   var      kw   var   var
```

which means “if THEN then THEN := ELSE else ELSE := IF”. Whether THEN is a keyword depends on its *position in the statement*, which only the parser knows. A lexer cannot decide with priority rules. The usual fixes are:

- the lexer returns a generic IDENT and the grammar accepts identifiers wherever a keyword may appear; or
- lexer–parser feedback**: the parser tells the lexer which reading it expects next.

In the real world. C has the same disease in a milder form. In `T * x;` the token `T` is a type name if a `typedef` declared it, and then the line declares a pointer. Otherwise it is a variable, and the line is a multiplication. C compilers solve it with the “lexer hack”: the lexer consults the symbol table, which the parser fills, to classify each identifier. Python 3.10’s `match` and C#’s `var` are “soft” keywords, handled the PL/I way.

6.4 Python: indentation is part of the syntax

In Python, blocks are defined by indentation, not by braces. The slide’s example:

```
if w == z:
    a = b
else:
    e = f
g = h
```

The parser needs to see where blocks open and close, so the lexer *invents* tokens that correspond to no characters at all: `INDENT` when the indentation increases and `DEDENT` when it decreases. Python’s own tokenizer (module `tokenize`) produces, for these five lines:

```
if w == z : NEWLINE INDENT a = b NEWLINE DEDENT else : NEWLINE
INDENT e = f NEWLINE DEDENT g = h NEWLINE ENDMARKER
```

To the parser, `INDENT` and `DEDENT` are exactly an opening and a closing brace.

Mechanism: The indentation stack

Keep a stack of indentation widths, initially `[0]`. Skip blank lines and comment-only lines. For every other line, let w be its number of leading spaces:

- if $w > top$: push w and emit `INDENT`;
- if $w < top$: pop and emit one `DEDENT` per pop until $top \leq w$; then, if $top \neq w$, report an *inconsistent dedent* error;
- if $w = top$: emit nothing.

At end of file, emit one `DEDENT` for each entry above 0 that is still on the stack.

Worked example: Two blocks closing at once

Line	Text	Stack after	Emitted
1	<code>if a:</code>	<code>[0]</code>	—
2	<code> if b:</code>	<code>[0, 4]</code>	<code>INDENT</code>
3	<code> x = 1</code>	<code>[0, 4, 8]</code>	<code>INDENT</code>
4	<code> y = 2</code>	<code>[0]</code>	<code>DEDENT DEDENT</code>

Python’s tokenizer agrees: `... x = 1 NEWLINE DEDENT DEDENT y = 2`
`....` Had line 4 been indented by 2 spaces, the stack would pop 8 and

then 4 and stop at $0 < 2$. Since $0 \neq 2$, that is an inconsistent dedent.

Cost. Each DEDENT pops an entry that some earlier INDENT pushed, and each line pushes at most once. Over the whole file, therefore,

$$\#DEDENT \leq \#INDENT \leq \#lines.$$

A single line may emit many DEDENTs, but the stack work is $O(1)$ per line *amortised*.

Proposition 6.1 (No finite automaton can lex indentation). *No finite-state machine that reads the source and outputs tokens can produce the correct INDENT and DEDENT tokens for every file.*

Proof. Suppose a machine with N states did. For $d = 1, 2, \dots, N + 1$, let p_d be the prefix made of the d lines

$$^1 \mathbf{x}: \quad ^2 \mathbf{x}: \quad \dots \quad ^d \mathbf{x}:$$

(line j has j leading spaces). After p_d the correct stack is $[0, 1, \dots, d]$. Now append the suffix line $\mathbf{y}$ with no indentation. The correct output for that line contains exactly d DEDENTs.

There are $N + 1$ prefixes and only N states. By the pigeonhole principle, two prefixes p_i and p_j with $i < j$ leave the machine in the same state. A finite-state machine's future output depends only on its current state and the remaining input. So on the same suffix $\mathbf{y}$ it produces the same output after p_i as after p_j . But the correct outputs contain $i \neq j$ DEDENTs, a contradiction. $\square$

Key insight. This is the same reason balanced parentheses are not regular: the machine must count to an unbounded depth. The Python lexer is a DFA *plus a stack*. The DFA handles the characters; a few lines of extra code handle the stack.

6.5 The general escape hatch: lexer states

All three quirks are handled the same way: give the lexer a little more memory than one DFA state. That memory can be lookahead (Fortran), information from the parser (PL/I, C), or a stack (Python). **flex** provides a structured version of this idea, *start conditions*: the lexer has modes, and each rule is active only in certain modes (Section 8).

7 Writing a Lexer by Hand

7.1 Two ways to build a lexer

In practice, and for the course project, there are two ways:

- write an **ad-hoc** analyser by hand;
- use an **automatic tool** such as **lex** or **flex**.

The slide says that the first is usually more efficient but tedious, and that the second is less efficient but more portable. The efficiency gap is smaller than it sounds, since a generated DFA scanner is already very fast. Production compilers that hand-write their lexers (Clang, GCC's C preprocessor library, Go, Rust) do so mostly for *control*: precise error messages, context tricks like C's typedef problem, Unicode handling and incremental re-lexing in editors.

The hand-written scanner below comes from the Stanford Dragon-book lecture notes, as shown on slides 85 to 90. We assemble it into one program, run it, and then read it as an automaton.

7.2 The token codes

```
#define T_SEMICOLON ';' // use ASCII values for single char tokens
#define T_LPAREN '('
#define T_RPAREN ')'
#define T_ASSIGN '='
#define T_DIVIDE '/'
...
#define T_WHILE 257 // reserved words
#define T_IF 258
#define T_RETURN 259
...
#define T_IDENTIFIER 268 // identifiers, constants, etc.
#define T_INTEGER 269
#define T_DOUBLE 270
#define T_STRING 271

#define T_END 349 // code used when at end of file
#define T_UNKNOWN 350 // token was unrecognized by scanner
```

Concept: Why these numbers

A one-character token uses its own ASCII code as its token number, so ';' is token 59. No name needs inventing, and the scanner can simply return the character it read. Every other token starts at 257, above all 8-bit character codes (0 to 255), so the two ranges can never collide. `yacc` and `bison` use the same convention, which is why such a scanner can be plugged straight into a generated parser. `T_END` and `T_UNKNOWN` play the roles of “end of input” and the catch-all of Section 4.

7.3 The token structure

```
struct token_t {
    int type; // one of the token codes from above
    union {
        char stringValue[256]; // holds lexeme value if string/identifier
        int intValue; // holds lexeme value if integer
        double doubleValue; // holds lexeme value if double
    } val;
};
```

This is the pair $\langle \text{name}, \text{attribute} \rangle$ of Section 1, written in C. The `union` makes its three members share the same memory, so the union is as large as its largest member (256 bytes) and only one member is meaningful at a time. The `type` field tells which one: `stringValue` for identifiers and strings, `intValue` for integers, `doubleValue` for doubles. This pattern is called a *tagged union*.

The driver loop and the initialisation are short:

```
int main(int argc, char *argv[]) {
    struct token_t token;
    InitScanner();
    while (ScanOneToken(stdin, &token) != T_END)
        ; // this is where you would process each token
    return 0;
}

static void InitScanner() {
    create_reserved_table(); // maps reserved words to token types
    insert_reserved("WHILE", T_WHILE);
    insert_reserved("IF", T_IF);
    insert_reserved("RETURN", T_RETURN);
    ...
}
```

In a real compiler the parser replaces `main`: it calls `ScanOneToken` whenever it needs the next token. Keywords are recognised by *table lookup*, the second option of the trade-off box in Section 4.

7.4 ScanOneToken, read as an automaton

Here are slides 88 to 90 joined together. The “...” on the slides stand for the remaining case labels.

```
1 static int ScanOneToken(FILE *fp, struct token_t *token) {
2     int i, ch, nextch;
3     ch = getc(fp); // read next char from input stream
4     while (isspace(ch)) // if necessary, keep reading til non-space
5         ch = getc(fp); // (discard any white space)
6     switch (ch) {
7         case '/': // could either begin comment or T_DIVIDE
8             nextch = getc(fp);
9             if (nextch == '/' || nextch == '*')
10                ; // here you would skip over the comment
11            else
12                ungetc(nextch, fp); // fall-through to single-char token case
13        case ';': case ',': case '=': // ... and other single char tokens
14            token->type = ch; // ASCII value is used as token type
15            return ch;
16        case 'A': case 'B': case 'C': // ... and other upper letters
17            token->val.stringValue[0] = ch;
18            for (i = 1; isupper(ch = getc(fp)); i++) // gather uppercase
19                token->val.stringValue[i] = ch;
20            ungetc(ch, fp);
21            token->val.stringValue[i] = '\0';
22            token->type = lookup_reserved(token->val.stringValue);
23            return token->type;
24        case 'a': case 'b': case 'c': // ... and other lower letters
```

```

25     token->type = T_IDENTIFIER;
26     token->val.stringValue[0] = ch;
27     for (i = 1; islower(ch = getc(fp)); i++)
28         token->val.stringValue[i] = ch;           // gather lowercase
29     ungetc(ch, fp);
30     token->val.stringValue[i] = '\0';
31     if (lookup_syntab(token->val.stringValue) == NULL)
32         add_syntab(token->val.stringValue);       // get symbol for ident
33     return T_IDENTIFIER;
34 case '0': case '1': case '2': case '3':           // ... and other digits
35     token->type = T_INTEGER;
36     token->val.intValue = ch - '0';
37     while (isdigit(ch = getc(fp)))                // convert digits to number
38         token->val.intValue = token->val.intValue * 10 + ch - '0';
39     ungetc(ch, fp);
40     return T_INTEGER;
41 case EOF:
42     return T_END;
43 default:                                           // anything else is not recognized
44     token->val.intValue = ch;
45     token->type = T_UNKNOWN;
46     return T_UNKNOWN;
47 }
48 }

```

How to read it:

- **Lines 3–5** discard blanks, which is job 4 of the lexer.
- **Line 6:** the `switch` on the first non-blank character plays the role of the *start state* of a DFA. Each `case` is one outgoing transition. This works because the token classes were designed to be told apart by their first character: uppercase starts a keyword, lowercase an identifier, a digit a number.
- **Lines 7–12:** / needs **one character of lookahead**. The scanner reads the next character with `getc`. If it does not start a comment, `ungetc` pushes it back. There is no `break`, so control falls through into the single-character case, which returns `'/'`.
- **Lines 16–23:** the `for` loop is a DFA state with a self-loop on uppercase letters. It runs as long as the next character fits, which is longest match. The character that ends the loop has been read, so `ungetc` pushes it back: this is the “stuck character is pushed back” rule of Section 4. Then the keyword table decides which keyword it is.
- **Lines 24–33:** identifiers are handled the same way, plus job 3 of the lexer, the symbol-table update.
- **Lines 34–40:** digits compute the value on the fly (Horner’s rule): reading 407 produces 4, then 40, then 407.
- **Lines 41–46:** EOF becomes `T_END`, and `default` is the catch-all.

Scanner concept	In this C code
DFA start state	<code>switch (ch)</code>
transition out of the start state	one <code>case</code> label
self-loop on a character class	<code>for/while</code> with <code>isupper</code> , <code>islower</code> , <code>isdigit</code>
longest match	loops run as long as the next character fits
push back the stuck character	<code>ungetc(ch, fp)</code>
token label	the returned code
catch-all rule	<code>default</code> : returning <code>T_UNKNOWN</code>

Worked example: Running it

Assembled with small stand-ins for the tables and a `main` that prints each token, the scanner was run on the two lines

```
WHILE (count) x = x / 2; // halve
IF (x) RETURN y12; WHILEX
```

and printed (real output):

```
T_WHILE '(' ID(count) ') ' ID(x) '=' ID(x) '/' INT(2) ';' '/' ID(halve)
T_IF '(' ID(x) ')' T_RETURN ID(y) INT(12) ';' T_UNKNOWN
```

Three things deserve a second look. The comment came out as `'/'` followed by `ID(halve)`. The name `y12` became two tokens. And `WHILEX` became `T_UNKNOWN`. All three are defects of the slide's code, explained next.

Debugging: Six defects in the slide's scanner, and their fixes

1. **Comments are not skipped, and a stray `'/'` is returned.** After “here you would skip over the comment” there is no `return` and no `break`. So control falls through into the single-character case and returns `'/'`. The second `/` has been consumed, and the comment text is then scanned as code. The fix is to skip the comment and restart:

```
if (nextch == '/') {                                // line comment
    while ((ch = getc(fp)) != '\n' && ch != EOF) ;
    return ScanOneToken(fp, token);                  // restart after it
} else if (nextch == '*') {                          // block comment
    int prev = 0;
    while ((ch = getc(fp)) != EOF && !(prev == '*' && ch == '/')) prev =
        ch;
    return ScanOneToken(fp, token);
} else ungetc(nextch, fp);
```

With this fix the output above loses `'/'` `ID(halve)`, and nothing else changes (checked).

2. **Identifiers are lowercase only.** `y12` gives `ID(y)` followed by `INT(12)`, and `aB` gives `ID(a)` followed by a keyword lookup of `B`. This is a deliberate simplification of the toy language (keywords uppercase, identifiers lowercase), but it is not how real languages work.
3. **Non-keywords in uppercase.** `WHILEX` is not in the reserved table, and the slide does not say what `lookup_reserved` returns then. Here

it returned `T_UNKNOWN`.

4. **Buffer overflow.** `stringValue[256]` is filled with no bound check, so a 300-letter identifier writes past the array. Add `i < 255` to the loop condition.
5. **Integer overflow.** `intValue * 10 + ...` silently overflows on long digit strings. Compare with `INT_MAX / 10` before multiplying.
6. **Only one character of pushback.** The C standard guarantees that `ungetc` can push back just *one* character. That is enough here, since every decision needs one character of lookahead. It is not enough for floats: `3e-y` needs three characters pushed back (Section 5). A scanner for such tokens keeps its own input buffer, with a pointer that it can move back.

Technique: Peek, then commit or push back

Every multi-character operator follows the same pattern:

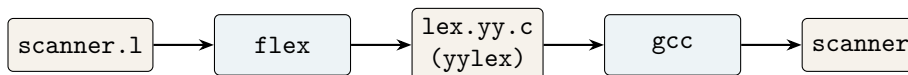
```
case '<':
    nextch = getc(fp);
    if (nextch == '=') return T_LE;      // commit: "<=" is one token
    ungetc(nextch, fp);                 // push back: it was just "<"
    return '<';
```

Peek at the next character. If it extends the token, consume it. If not, push it back. This is longest match with one character of lookahead.

8 Generating a Lexer with flex

8.1 What flex does

`flex` is a free implementation of the classic Unix tool `lex`. It implements this chapter and Chapter 3 end to end:



Inside the `flex` box:

1. it reads the regular expressions;
2. it builds the combined NFA, with the accepting states labelled by rule;
3. it converts that NFA to a DFA with the subset construction;
4. it minimises the DFA as much as possible;
5. it writes C code that runs the DFA from compressed tables;
6. that code resolves conflicts by the longest matching prefix, then by rule order.

You can watch steps 2 to 4 happen. `flex -v` prints statistics, for example for `count.l` below:

```
$ flex -v count.l
```

```
15/2000 NFA states
7/1000 DFA states (20 words)
3 rules
Compressed tables always back-up
...
```

“Back-up” is flex’s word for the pushback of Section 4: some DFA states may be passed through on the way to a longer match that never comes.

8.2 The input file

A flex file has up to four parts, separated by %% lines:

```
%{
Declarations      /* C code, copied verbatim to the top of lex.yy.c */
}%
Definitions      /* options and named patterns (regular definitions) */
%%
Rules            /* pattern  action */
%%
User subroutines  /* C code, copied verbatim to the end of lex.yy.c */
```

- **Declarations:** #includes, global variables, token constants.
- **Definitions:** lines `name pattern`, used later as `{name}`, and options such as `%option noyywrap`.
- **Rules:** the heart of the file, described next.
- **User subroutines:** helper functions, and often `main`.

8.3 Rules, and how flex chooses

Each rule has the form `pattern action`. The pattern is a regular expression and the action is a block of C code. Each time `yylex()` is called:

1. at the current position, find the longest prefix matched by *any* rule;
2. if several rules match that same longest prefix, take the one written **first**;
3. set `yytext` to the lexeme and `yylen` to its length, then run the action;
4. if the action executes `return`, `yylex` returns that value to its caller (normally the parser). Otherwise it continues scanning.

```
[0-9]+  {printf("This is a number");}
[a-z]+  {printf("This is symbol");}
```

The default rule. If no rule matches the current character, flex copies that character to the output unchanged, as if a hidden `.\|n ECHO;` rule came last. This is convenient for filters, but a scanner for a compiler should never print silently. Use `%option nodefault` (or `flex -s`) to turn an unmatched character into an error, or write your own catch-all rule.

8.4 Pattern syntax

Syntax	Meaning	Example
[], -, +, *, ?	as in Section 2	[a-z0-9]+
.	any character except newline	"#".* (a line comment)
[^x]	any character not in the set, newline included	[^0-9]
x{n,m}	<i>x</i> repeated <i>n</i> to <i>m</i> times	[0-9]{1,3}
"x"	<i>x</i> literally, even with special characters	"x*", "/"
\x	escape a single special character	\., \n
{name}	the pattern defined in the definitions part	{digit}+
r/s	<i>r</i> , only if followed by <i>s</i> (<i>s</i> not consumed)	the Fortran rule of Section 6
^r, r\$	<i>r</i> at the beginning / end of a line	^#.*
<SC>r	<i>r</i> , only in start condition SC	<COMMENT>.
<<EOF>>	end of file	

Warning / Gotcha: Three classic slips

- . does *not* match a newline, but a complemented class like [^*] *does*. Comment patterns depend on this difference (Section 9).
- An unquoted space ends the pattern. To match a blank, write " ", \ or [].
- A rule must start in column 1. An indented line in the rules section is treated as C code and copied to the output.

8.5 Talking to the scanner

Interface / API: The yy interface

- int `yylex(void)` scans tokens from `yyin` until the end of the file, or until an action executes `return`.
- FILE `*yyin` is the input file, `stdin` by default. FILE `*yyout` is where `ECHO` writes, `stdout` by default.
- char `*yytext` is the null-terminated text of the lexeme just matched, and int `yylen` is its length.
- `yyval` holds the attribute of the token, so the parser receives the pair (`yylex()`'s return value, `yyval`).
- `yyloc` holds the location of the token (line and column), when locations are enabled.
- `yylineno` is the current line number, available with `%option yylineno`.
- `ECHO` copies `yytext` to `yyout`. `yyless(n)` pushes back all but the first `n` characters of the lexeme, which is `flex`'s own pushback.
- int `yywrap(void)` is called at end of file. Returning 1 means "no more

input”.
yylval and yylloc are normally declared by bison (through %union and %locations). Without bison you declare them yourself.

Warning / Gotcha: -ll, -lfl or noyywrap?

The lex library supplies a default main (which just calls yylex()) and a default yywrap (which returns 1). On macOS it is linked with -ll; on Linux with -lfl. The slides write “gcc ... lex.yy.c ll”, but the dash got lost: it is -ll. If you write %option noyywrap and your own main, no library is needed at all.

8.6 Example 1: hiding numbers

```
%%  
[0-9]+ printf("?");  
.      ECHO;
```

```
$ flex hide-digits.l           # produces lex.yy.c  
$ gcc -o hide-digits lex.yy.c -ll  
$ printf 'call 0612345678 at 9am\nroom B12\n' | ./hide-digits  
call ? at ?am  
room B?
```

Each maximal run of digits becomes one ?, because [0-9]+ with longest match swallows the whole run. Every other character is echoed by the second rule. But no rule matches the newline, since . excludes it. So who printed the line break? The hidden default rule did. The debugging switch flex -d shows this (real trace on input a12 followed by a newline):

```
--accepting rule at line 3 ("a")  
--accepting rule at line 2 ("12")  
--accepting default rule ("  
")
```

8.7 Example 2: counting like wc

```
%{  
int numChars = 0, numWords = 0, numLines = 0;  
%}  
%%  
\n      {numLines++; numChars++;}  
[^\t\n]+ {numWords++; numChars += yyleng;}  
.      {numChars++;}  
%%  
int main() {  
    yylex();  
    printf("%d\t%d\t%d\n", numChars, numWords, numLines);  
}
```

- A **word** is a maximal run of non-blank characters, `[^\t\n]+`. Longest match makes the rule take the whole word at once, and `yyleng` adds its length to the character count.
- `\n` counts a line, and a character.
- `.` counts every other character (the spaces and tabs).

On the input `hi there\nyo\n` it prints `12 3 2`, the same counts as `wc` (which prints them in the order lines, words, characters).

Rule order matters here, and it is not obvious why. On a one-letter word such as `a`, the word rule and `.` both match exactly one character. That is a tie, and the earlier rule wins. If you swap the last two rules, every one-letter word is counted as a mere character. On `a b cd` the swapped version reports 1 word instead of 3 (checked).

8.8 Example 3: a typical compiler front end

Slides 98 and 99 show the textbook's scanner for a small language. Here it is, typed out:

```
%{
/* definitions of manifest constants
   LT, LE, EQ, NE, GT, GE, IF, THEN, ELSE, ID, NUMBER, RELOP */
%}
/* regular definitions */
delim    [ \t\n]
ws       {delim}+
letter   [A-Za-z]
digit    [0-9]
id       {letter}{letter}|{digit}*
number   {digit}+(\.{digit}+)?(E[+-]?{digit}+)?
%%
{ws}      { /* no action and no return */ }
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"      {yylval = LT; return(RELOP);}
"<="     {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"     {yylval = NE; return(RELOP);}
">"      {yylval = GT; return(RELOP);}
">="     {yylval = GE; return(RELOP);}
%%
int installID() { /* install the lexeme (yytext, length yyleng) into the
                  symbol table and return a pointer to its entry */ }
int installNum() { /* similar, but numerical constants go to their own table
                  */ }
```

Every idea of this chapter appears in it:

- `{ws}` has an empty action and no `return`, so blanks are matched and thrown away (job 4).

- The keywords come *before* {id}, so if is IF by priority, while ifx is an ID by longest match.
- {id} calls installID, which updates the symbol table (job 3), and passes the entry through yylval (the attribute).
- The six relational operators share one token RELOP, distinguished by the attribute, as recommended in Section 1.
- "<=" and "<" need no ordering: on <=, longest match picks the two-character rule.

With the constants filled in, simple tables behind installID and installNum, and a main that prints each token, this is the real output (each run prints one line, wrapped here):

```
$ echo 'if x1 <= 3.14E-2 then y else ifx <> x1' | ./typical
<IF> <ID,0:x1> <RELOP,LE> <NUMBER,0:3.14E-2> <THEN> <ID,1:y> <ELSE>
<ID,2:ifx> <RELOP,NE> <ID,0:x1>
$ echo 'if a>=b then c=1.5E3 else d=2.e5' | ./typical
<IF> <ID,0:a> <RELOP,GE> <ID,1:b> <THEN> <ID,2:c> <RELOP,EQ> <NUMBER,0:1.5E3>
<ELSE> <ID,3:d> <RELOP,EQ> <NUMBER,1:2> .<ID,4:e5>
```

Both occurrences of x1 got symbol-table index 0, which is the “same pointer” of Section 1. In the second run, 2.e5 is not a number for this pattern, because a fraction needs a digit after the point. So 2 becomes a NUMBER, the point matches no rule and is *echoed by the default rule* (the lone . in the output), and e5 becomes an identifier. A real compiler must catch that point with a catch-all rule.

Warning / Gotcha: (int) installID() on a 64-bit machine

On the slide, installID returns a pointer that is stored in an int. On today’s 64-bit machines a pointer does not fit in an int. Declare yylval as a union with a pointer member, which is what bison’s %union does.

8.9 Reporting line and column

Error messages need positions, and yylloc expects a line and a column. flex can count lines (%option yylineno) but not columns. The standard trick is the macro YY_USER_ACTION: flex executes it before *every* action, including the empty actions of skipped blanks and comments. It can therefore walk through yytext and advance the counters.

```
%option noyywrap
%{
int line = 1, col = 1;          /* position of the NEXT character */
int tline, tcol;               /* position of the current lexeme */
#define YY_USER_ACTION          \
    tline = line; tcol = col;    \
    for (int i = 0; i < yytext; i++) \
        if (yytext[i] == '\n') { line++; col = 1; } \
        else col++;
#define OUT(kind) printf("%d:%d %s %s\n", tline, tcol, kind, yytext)
%}
DIGIT  [0-9]
```

```
ID      [a-z][a-z0-9]*
%%
"(*([~*]"*"+[~*]))*"*"+")" { /* block comment, may span lines */ }
"#".* { /* line comment */ }
[ \t\n]+ { /* blanks */ }
let|print OUT("KEYWORD");
{ID} OUT("ID");
0|[1-9]{DIGIT}* OUT("INT");
":="|[-+*/] OUT("OP");
[();] OUT("PUNCT");
. printf("%d:%d error: unexpected '%s'\n",
    tline, tcol, yytext);
%%
int main(void) { yylex(); return 0; }
```

This scanner is for a small calculator language with Pascal-style (`* ... *`) comments; the comment pattern is derived in Section 9. On the three-line input

```
let x := 10; # set x
(* a comment
   on two lines *) print (x * 02) $
```

it prints one token per line. Here is the real output, arranged in columns to save space:

```
1:1 KEYWORD let      1:5 ID x      1:7 OP :=      1:10 INT 10
1:12 PUNCT ;        3:20 KEYWORD print 3:26 PUNCT (    3:27 ID x
3:29 OP *           3:31 INT 0      3:32 INT 2      3:33 PUNCT )
3:35 error: unexpected '$'
```

- `print` is correctly at line 3, column 20, even though the comment that ends on line 3 was skipped. `YY_USER_ACTION` also ran for the comment and counted its newline.
- `02` is two tokens, because the integer pattern `0|[1-9]{DIGIT}*` forbids leading zeros. Longest match takes `0` (no rule matches `02`), and then `2` on its own.
- `$` reaches the catch-all, which reports it and moves on.

8.10 Start conditions: modes instead of monster patterns

For comments and strings there is an alternative to one huge regular expression: switch the scanner into a *mode*. An exclusive start condition (%x) makes only the rules prefixed with its name active.

```
%x COMMENT
%{
int depth = 0;
}%
%%
"(" { depth = 1; BEGIN(COMMENT); }
<COMMENT>"(" { depth++; }
<COMMENT>"*)" { if (--depth == 0) BEGIN(INITIAL); }
<COMMENT>.\n { /* skip */ }
```

```
<COMMENT><<EOF>> { printf("[error: unterminated comment]"); return 0; }
```

With an identifier rule and a blank rule added, the full scanner was run on one line (real output):

```
input:  x (* a (* nested *) still *) y (* open
output: [ID:x][ID:y][error: unterminated comment]
```

Two things here are beyond a single regular expression. It reports an *unterminated* comment cleanly, and it supports *nested* comments by counting depth. Nesting is the balanced-parentheses language, which is not regular (Section 6); the counter is the extra memory.

8.11 Debugging a flex specification

Debugging: Tools and symptoms

- **flex -d** makes the scanner print every rule it accepts, as in Example 1. It is the fastest way to see which rule ate your input.
- **flex -v** prints NFA, DFA and table statistics. **flex -b** writes `lex.backup`, listing the DFA states that require backing up.
- **“warning, rule cannot be matched”** means that some earlier rule matches everything this rule matches, with at least the same length. The rule is dead code.
- **Characters appear in the output that no rule prints:** that is the default rule. Add a catch-all, or use `%option nodefault`.
- **A rule “does not fire”:** it is either indented (so it was copied as C code) or shadowed by an earlier rule of equal length.

Worked example: Predict the output

```
1 %option noyywrap
2 %%
3 a+b    printf("[X:%s]", yytext);
4 ab*    printf("[Y:%s]", yytext);
5 b      printf("[Z]");
6 ab     printf("[W]");
7 .|\n   ECHO;
```

Input: aaabbab, a newline, abbbca, a newline. Try it before reading on. At each position, list the length each rule would match. Take the longest; on a tie, take the earlier rule.

Remaining input	a+b	ab*	others	Winner
aaabbbab...	4 (aaab)	1 (a)	—	[X:aaab]
bab...	—	—	b: 1	[Z]
ab\n...	2	2	ab: 2	tie, first rule: [X:ab]
\n	—	—	.\n: 1	echo newline
abbbca...	2 (ab)	4 (abbb)	ab: 2	[Y:abbb]
ca...	—	—	.: 1	echo c
a\n	—	1	—	[Y:a], then echo newline

Real output:

```
predict.l:6: warning, rule cannot be matched
[X:aaab] [Z] [X:ab]
[Y:abbb] c [Y:a]
```

Rule **ab** (line 6) can never fire. Whenever it matches, **a+b** matches the same two characters and comes first. **flex** detects this at generation time and warns.

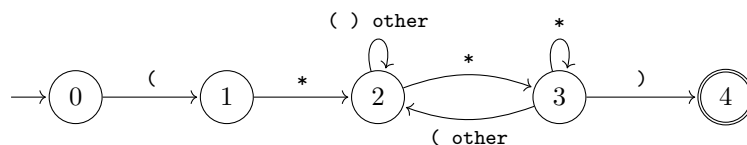
9 Tricks, Techniques and Classic Problems

Technique: Exam-speed recipes

- **Thompson counts (merged convention):** $|Q| = 2(\text{leaves} + \#| + \#*) - \#\text{concatenations}$ and $|E| = \text{leaves} + 4(\#| + \#*)$. With Chapter 3's ε -edge convention, drop the “ $-\#\text{concatenations}$ ” and add one edge per concatenation. Always parse first.
- **NFA simulation by hand:** write S as a sorted set after every character, and close it under ε after every move. Accept if and only if the final set contains an accepting state.
- **Longest match by hand:** keep (label, end) of the last accepting step. When stuck, emit it, push back everything read after it, and restart there. Rule order is consulted only for equal lengths.
- **Counting reads:** count every character fetched, including the one that gets the scanner stuck, and count re-reads after a pushback again. Decide explicitly whether reaching end of file counts (usually it does not).
- **Maximum pushback of a specification:** 1+ the longest run of non-accepting DFA states that can follow an accepting one. A reachable cycle of non-accepting states means the pushback is unbounded.
- **Delimited tokens (comments, strings):** draw the small DFA first, then read off the regular expression (below).
- **“Needs many states” or “not regular”:** exhibit many prefixes that must lead to different states, and separate any two of them with one suffix (Section 5.4), or use pigeonhole (Section 6).

9.1 Deriving a comment pattern from its DFA

Block comments are the classic hard pattern, because the closing delimiter must be the *first* one after the opening delimiter. Do not guess the regular expression. Build the DFA, then read the expression off it. We do it for Pascal-style comments (`* ... *`), which the scanner of Section 8 uses. The alphabet splits into four classes: `(`, `*`, `)` and *other*, where *other* includes newline.



State	What it remembers
0	nothing read yet
1	read the <code>(</code> of the opener
2	inside the comment, and the last character was not <code>*</code>
3	inside the comment, and the last character was <code>*</code> (a closer may be starting)
4	read the closer; the comment is finished

Every missing transition goes to a dead state, so there are six states in all. Every one of them is needed. State 4 is the only accepting state, the dead state is the only one from which nothing can be accepted, and the other four need different shortest completions (`(**)`, `**`), `*)`, `*)`). So this DFA is minimal.

Reading off the expression. After the opener `(*`, the machine circulates between states 2 and 3. From state 2 it can stay with one non-`*` character, `[^*]`, or make a round trip `2 → 3 → 2` reading a run of stars and then a character that is neither `*` nor `)`, that is `"*" + [^*)]`. It leaves through a run of stars and the closing `)`. Hence

`"(*" ([^*] | "*" + [^*)]) * "*" + ")"`.

For C comments `/* ... */`, replace the parenthesis by `/` in the closer, and in the class that may not follow a run of stars:

`"/*" ([^*] | "*" + [^*/]) * "*" + "/"`.

In `flex`, `[^*]` also matches newline, so both patterns accept comments that span several lines. Here is the C version run by `flex` on single strings. In the last column, a bracketed part is what the comment rule matched, and the rest fell to a one-character rule:

Input	Whole input is a comment?	Real scanner output
/**/	yes	[/**/]
/***/	yes	[/***/]
/*/	no: the / after /* is inside, so no closer	/ * / as three characters
/* a * b **/	yes: * b is a star followed by a non-/	[/* a * b **/]
/* a */ b */	no: the comment ends at the first */	[/* a */] then <code>␣b ␣*</code> /
/* x */*/	no, same reason	[/* x */] then <code>* /</code>

Warning / Gotcha: The tempting rule "(\".*\")"

Run with an identifier rule, a blank rule and a one-character fallback, this rule gives (real output):

```
input:  x (* c1 *) y (* c2 *) z
output: [ID:x][COMMENT:(* c1 *) y (* c2 *)][ID:z]
input:  x (* one          (newline)      two *) y
output: [ID:x][?( ) [?*][ID:one][?\n][ID:two][?*][?)] [ID:y]
```

- **Two comments on one line:** `.*` can run over the first `*)`, and longest match makes it run as far as possible, to the *last* `*)` on the line. The code `y` between the comments disappears.
- **A comment over two lines:** `.` cannot cross the newline, so the rule never matches, and the delimiters leak out as junk tokens.

Moving the rule earlier or later in the file does not help. Rule order only breaks ties between equal lengths, and here the problem is the *length* itself.

9.2 The quadratic pushback trap

Chapter 3 showed that longest match can re-read the input quadratically many times. Here is the cleanest family to count by hand. Take the rules $X = ab$ and $Y = (ab)^*c$, and the input $(ab)^k$ with no `c` anywhere.

- **Token 1.** After `ab`, X accepts, but Y is still alive, hoping for a `c`. The scanner reads all $2k$ characters, reaches end of file, and falls back to `ab`. It has made $2k$ reads and pushes back $2k - 2$ characters.
- **Token j .** The same happens from position $2(j - 1)$: $2(k - j + 1)$ reads, and $2(k - j)$ characters pushed back.

In total,

$$\sum_{j=1}^k 2(k - j + 1) = 2 \cdot \frac{k(k + 1)}{2} = k(k + 1) \text{ reads}$$

for an input of $n = 2k$ characters. That is $\Theta(n^2)$, with a single token pushing back as many as $2k - 2 = n - 2$ characters. A simulation script confirms $k(k + 1)$ for $k = 1, \dots, 6$.

The fix is Reps' memoisation (Chapter 3). When backtracking passes through a DFA state q at input position i , record that the pair (q, i) leads to no accepting state. The next token then stops as soon as it reaches a pair that the

previous token already proved hopeless. Here, every token after the first stops after three reads (two for the last one): it reads *a*, *b*, *a*, and lands on a pair that token 1 marked. The total drops from $k(k+1)$ to $2k+3(k-2)+2 = 5k-4$ reads, which is linear (checked by simulation). The tokens are unchanged, because the reads that were skipped could only have ended in the same backtrack.

9.3 Classic problems

Worked example: Short questions with answers

P1. How many states and edges does the merged-convention Thompson NFA for $(ab \mid c)^*$ have?

Parse: $((a \cdot b) \mid c)^*$, with 3 leaves, 1 alternation, 1 star and 1 concatenation. So $|Q| = 2(3 + 1 + 1) - 1 = 9$ and $|E| = 3 + 4 \cdot 2 = 11$. With Chapter 3's convention, it has 10 states and 12 edges.

P2. A Thompson NFA has 40 states. At most how many edges does one simulation step scan?

At most 80, since by (P3) each state has at most two outgoing edges. That makes one step $O(|Q|)$.

P3. Rules in this order: $[a-z]^+$ for ID, then "if" for IF. Input if. Token?

ID. The two matches have the same length, so the earlier rule wins. *flex* warns that the IF rule cannot be matched.

P4. Rules INT $0 \mid [1-9][0-9]^*$ and ID $[a-z]^+$. Input 0x1F. Tokens?

INT 0, ID *x*, INT 1, and then F matches nothing (catch-all or default rule). Add a rule HEX $0[xX][0-9a-fA-F]^+$, and longest match picks the 4-character HEX over the 1-character INT, wherever the rule is placed.

P5. Rules NUM $[0-9]^+$, FLT $[0-9]^+\backslash\.[0-9]^+$ and DOT $\backslash\.$. What is the maximum pushback, and on which input?

After an accepting NUM state, the only non-accepting state is “digits and a point, waiting for a digit”. One step, plus the stuck character, gives a maximum of 2. For example, 3.*x* gives NUM 3 and pushes back .*x*, then DOT, then whatever *x* is.

P6. Indentation widths 0, 4, 8, 2 on four consecutive lines. What happens at line 4?

The scanner pops 8 and emits DEDENT, then pops 4 and emits DEDENT. Now the top is $0 < 2$ and $0 \neq 2$, so it reports an inconsistent dedent.

P7. Why does the catch-all $\cdot$ not catch a stray newline in *flex*?
Because $\cdot$ excludes newline. Write $\cdot|\backslash n$ instead.

P8. Which C library call implements “push back the stuck character”, and what is its limit?

ungetc. Only one character of pushback is guaranteed.

9.4 Map to Problem Set 4

Problem	Where this chapter prepares it
1. Comment regex, greedy rule, minimal DFA, numeric literals	comment DFA and patterns, and the greedy-rule warning (this section); the sign trap and the float pattern (§2); the leading-zero integer rule (§8, line and column example)
2. Thompson size and structure, simulation cost	Proposition 3.2, Proposition 3.3, Proposition 3.5 (§3)
3. Maximal munch, pushback, quadratic trap	the ifu26=60 trace (§4); the 3e-y trace and the pushback bound (§5); the quadratic trap (this section); Reps' fix (Chapter 3)
4. Exponential DFAs	speed versus memory (§5.4); subset construction (§5); Chapter 3's blow-up proposition
5. Indentation, Fortran	§6
6. flex: predict and write	the predict-the-output example, YY_USER_ACTION, start conditions (§8); rule order and the default rule

Warning / Gotcha: Gotchas

- Longest match first, priority second. Never use rule order to “fix” a length problem.
- The character that gets the scanner stuck has been read. It must be pushed back, not lost.
- . excludes newline in flex; [^x] includes it.
- In flex, unmatched input is echoed silently by the default rule.
- Thompson's merged convention and Chapter 3's ε -edge convention give different counts. Say which one you use.
- A subset-construction DFA is usually not minimal (Section 5 had $s'_0 \equiv s'_2$, and $H \equiv H'$).

Notes on the original slides

- **Slides 49–50.** The token list for `while (ip < z) ++ip;` stops at the last `T_Ident`. The final `;` is also a token.
- **Slide 61.** “ $s \in Q$ is the initial state” should read $s_0 \in Q$. “ $\Delta \subset$ ” means $\Delta \subseteq$.
- **Slide 63.** The 8-state NFA for $(a \mid b)^*ac$ is not produced by the rules of slide 62: its initial state has an incoming edge, which contradicts property 2 printed on the same slide. Thompson's rules give 10 states (Section 3.3).
- **Slide 63, property 1.** With merged concatenation, the exact count is $2(\#operators + \#operands) - 3 \#concatenations$, so the bound is tight only for expressions without concatenation.
- **Slide 65.** “Output the token of the first one to get to an accepting state” must be read together with slide 68 (longest match). Taken literally, it

would cut `ifu26` after one character.

- **Slide 68.** “= would be read and then re-inserted” is exact for `ifu26=60` written without blanks. With the blanks of the slide’s input, the character read after `ifu26` is the blank.
- **Slide 73.** $O(|S||Q|^2)$ is the bound for arbitrary NFAs. For Thompson NFAs the cost is $O(|S||Q|)$ (Proposition 3.5).
- **Slide 80.** The DFA has 11 states, while the raw subset construction gives 12. The equivalent states $\{13\}$ and $\{14,15\}$ were merged into H. The exercise inputs also assume a blank rule and a catch-all that are not drawn, and `-` alone is not a token of that specification.
- **Slide 81.** Hopcroft’s $O(n \log n)$ assumes a fixed alphabet. In general it is $O(|\Sigma| n \log n)$.
- **Slide 88.** The comment placeholder falls through and returns `’/’` (Section 7, defect 1).
- **Slides 96–97.** `gcc ... lex.yy.c ll` should be `-ll` on macOS, or `-lfl` on Linux.
- **Slide 98.** `yylval = (int) installID()` truncates a pointer on 64-bit machines.

Summary: Chapter 4 in ten lines

1. The lexer cuts characters into lexemes, labels them with tokens $\langle \text{name}, \text{attribute} \rangle$, fills the symbol table, and drops blanks and comments. The parser pulls tokens one at a time.
2. Separate lexing from parsing for simplicity, efficiency and portability. It splits regular from context-free.
3. Choose token names by what the grammar must distinguish; put everything else in attributes.
4. Dragon-book Thompson merges states on concatenation: $|Q| = 2(\text{leaves} + \#| + \#*) - \#\text{concat}$. Its properties (one entry, one exit, at most two out-edges) make the merge safe and the simulation cheap.
5. NFA simulation is $S \leftarrow \varepsilon\text{-closure}(\text{move}(S, c))$: $O(|S||Q|^2)$ in general, $O(|S||Q|)$ for Thompson NFAs, $O(|S|)$ for a DFA.
6. Many tokens: merge the NFAs, label the accepting states, take the longest match, push back what was read beyond it, break ties by priority, and catch everything else with a lowest-priority one-character rule.
7. Subset construction precomputes the simulation. Labels go to the highest-priority token. DFAs can be exponential; Hopcroft minimises in $O(n \log n)$, but cannot remove a necessary blow-up.
8. Fortran needs lookahead, PL/I needs the parser, and Python needs a stack. Lexing is not always regular.
9. A hand-written scanner is a DFA in disguise: `switch` as the start state, loops as self-loops, `ungetc` as pushback, `default` as the catch-all.
10. `flex`: declarations, definitions, `%%` rules, `%%` code. Longest match then rule order, a default rule that echoes, `yytext/yylen/yylval`, `YY_USER_ACTION` for positions, start conditions for modes.